INF201- Deliverable 3

Introduction

The aim of this assignment is to develop an RPG battle simulator using object-oriented programming in Python. Tasks 1–3 focus on character management, dice, and weapons, while Tasks 4–5 introduce character subclasses and battle simulations.

The project demonstrates key concepts such as encapsulation, inheritance, and composition.

G2_2 Amin Elanbri and Filmon Tsegay Kahsay

link to github: https://github.com/filmonkah/Deliverable-3-INF201

Task1: Character Class

In [1]:
```
from Character import Character
```

In [2]:
```
# Creat a character with default HP
hero = Character("Hero")
print(hero.hp)
```

```
100
```

In [3]:
```
# Test damage reduction
hero.take_damage(30)
print(hero.hp)
```

```
70
```

In [4]:
```
# Test healing
hero.heal(20)
print(hero.hp)
```

```
90
```

In [5]:
```
hero.heal(25)
print(hero.hp)
```

```
100
```

In [6]:
```
print(hero.is_alive)
```

```
True
```

In [7]:
```
hero.take_damage(150)
print(hero.hp)
print(hero.is_alive)
```

```
0
False
```

In [8]:
```
# Test attacking another character
hero = Character("Hero")
enemy = Character("Enemy")

hero.attack(enemy)

print(hero.hp)
print(enemy.hp)
```

```
100
90
```

Results and discussion

The updated Character class successfully handles damage, healing, and attacks. On the test we did the code confirmed that HP stays between 0 and maximum HP, and that the is_alive property correctly identifies whether a charackter is alive.

Task 2: Dice Class

In [9]:
```
from Dice import Dice


d6 = Dice()
d20 = Dice(sides=20)

print(d6)
print(d20)
```

```
Dice(sides=6)
Dice(sides=20)
```

In [10]:
```
# Test rolling a 6-sided and a 20-sided dice
print(d6.roll())
print(d20.roll())
```

```
3
4
```

In [11]:
```
# Test calling the dice object directly
print(d6())
print(d20())
```

```
5
18
```

In [12]:
```
# Test that a dice cannot have fewer than 2 sides
try:
    invalide_dice = Dice(sides=1)
except ValueError as error:
    print(error)
```

```
A dice must have at least 2 sides.
```

Discussion and Conclusion

As you can see we have successfully generated random numbers for dice with different numbers of sides. Our test confirmed that both roll() and__call__() work as expected, and that invalid dice with fewer than twe sides raise a ValueError.

Task 3: Weapon Class:

In [13]:
```
from Dice import Dice
from items import Weapon
from Character import Character
```

In [14]:
```
# Attack with a weapon and check the damage taken by Enemy
sword = Weapon("Sword", Dice(6))
hero = Character("Hero", weapon=sword)
enemy = Character("Enemy")

hero.attack(enemy)

print(enemy.hp)
```

```
98
```

In [15]:
```
# Test attacking without an equipped weapon
hero = Character("Hero")
enemy = Character("Enemy")

hero.attack(enemy)

print(enemy.hp)
```

```
90
```

Discussion and Conclusion

We have successfully made the weapon class to use the Dice to calculate random damage. Based on oure test we can confirme a character equipped with a weapon deals damage based on a dice roll, while a character without a weapon uses the deafult damage of 10.

Task 4: Character specialization

In [16]:
```
from Character import Character, Mage, Warrior
from Dice import Dice
from items import Weapon
```

In [17]:
```
#here I`m creating a warrior and showing it`s attributes
warrior = Warrior("Geir")
print(repr(warrior)) #shows name, Hp, damage and armor
print(f"Is a Character: {isinstance(warrior, Character)}")

#testing that the armor reduces incoming damage
warrior.take_damage(11) #11-5 armor = 8 damage
print(f"After 11 damage: {warrior.hp} HP")
warrior.take_damage(4)#4 - 5 armor is below 0, so no damage
```

```
print(f"After 4 damage: {warrior.hp} HP")
warrior.take_damage(200) #more damage than the warrior has HP
print(f"After 200 damage: {warrior.hp} HP, alive: {warrior.is_alive}")
```

```
<Warrior name = Geir hp=100 damage=10 armor=5>
Is a Character: True
After 11 damage: 94 HP
After 4 damage: 94 HP
After 200 damage: 0 HP, alive: False
```

In [18]:
```
#mage without weapon
#creating a mage and an enemy, and letting the mage attack five times
mage = Mage("Kari")
enemy = Character("Enemy")
print(repr(mage))

for attack_number in range(1, 6): #five attacks in a row
    mage.attack(enemy) #spell if mane >= 10, otherwise base attack
    print(f"Attck {attack_number}: enemy HP = {enemy.hp}, mage mana = {mage.mana
```

```
<Mage name = Kari hp=100 damage=10 mana=30/30>
Attck 1: enemy HP = 80, mage mana = 20
Attck 2: enemy HP = 60, mage mana = 10
Attck 3: enemy HP = 40, mage mana = 0
Attck 4: enemy HP = 30, mage mana = 5
Attck 5: enemy HP = 20, mage mana = 10
```

In [29]:
```
#mage with weapon
#testing that spells ignore the weapon, but base attacks use it
staff = Weapon("Staff", Dice(6)) # weapon that deals 1-6 damage
mage = Mage("kari", weapon=staff)
enemy = Character("Enemy") # new enemy eith 100HP

mage.attack(enemy) #the first attack is a spell
assert enemy.hp == 80 # spell deals double damage 2*10, not weapon damage
mage.attack(enemy) #second spell
mage.attack(enemy) #third spell, mana = 0
print(f"After 3 spells: enemy HP = {enemy.hp}, mage = {mage.mana}")

hp_before = enemy.hp #remember HP before the base attack
mage.attack(enemy) #not enough mana so base attack with the staff
Weapon_damage = hp_before - enemy.hp
assert 1 <= Weapon_damage <= 6
print(f"Base attack with staff dealt {Weapon_damage} damage, mana = {mage.mana}"
```

```
After 3 spells: enemy HP = 40, mage = 0
Base attack with staff dealt 2 damage, mana = 5
```

Results and discussion

The warrior`s armor redces 11 damage to 6 (94 HP), and 4 damage did nothing, since max() stops the armor from healing. after 200 damage it had 0 HPand was not alive.

The mage cast three spells of 20 damage, then used base attacks that restored 5 mana each, and after five attacks the enemy had 20 HP left. Wuth a stadd, spells still dealt 20 damage, while the base attack dealt 3, so spells ignore the weapon.

Taks 5: Running battles

In [20]:
```
from battle import Battle
from Character import Character, Mage, Warrior
from Dice import Dice
from items import Weapon
```

In [21]:
```
# Test play_round() on two normal characters
battle = Battle(Character("Hero"), Character("Enemy")) # two characters without
print(battle.play_round()) #first round
print(battle.play_round()) # second round
```

```
Round 1: Hero 90 HP, Enemy 90 HP
Round 2: Hero 80 HP, Enemy 80 HP
```

In [23]:
```
#Full battle between two normal characteres without weapons
battle = Battle(Character("Hero"), Character("Enemy")) # a new battle with full
winner = battle.simulate_battle() #play until one dies
for line in battle.log: #Show every round
    print(line)
print(f"Winner: {winner.name} after {battle.round_number} rounds")
```

```
Round 1: Hero 90 HP, Enemy 90 HP
Round 2: Hero 80 HP, Enemy 80 HP
Round 3: Hero 70 HP, Enemy 70 HP
Round 4: Hero 60 HP, Enemy 60 HP
Round 5: Hero 50 HP, Enemy 50 HP
Round 6: Hero 40 HP, Enemy 40 HP
Round 7: Hero 30 HP, Enemy 30 HP
Round 8: Hero 20 HP, Enemy 20 HP
Round 9: Hero 10 HP, Enemy 10 HP
Round 10: Hero 10 HP, Enemy 0 HP
Winner: Hero after 10 rounds
```

In [24]:
```
# Warrior against Mage without weapons
battle = Battle(Warrior("Geir"), Mage("Kari")) #warrior attacks first
winner = battle.simulate_battle()
for line in battle.log:
    print(line)
print (f"Winner: {winner.name} after {battle.round_number} rounds")
```

```
Round 1: Geir 85 HP, Kari 90 HP
Round 2: Geir 70 HP, Kari 80 HP
Round 3: Geir 55 HP, Kari 70 HP
Round 4: Geir 50 HP, Kari 60 HP
Round 5: Geir 45 HP, Kari 50 HP
Round 6: Geir 30 HP, Kari 40 HP
Round 7: Geir 25 HP, Kari 30 HP
Round 8: Geir 20 HP, Kari 20 HP
Round 9: Geir 5 HP, Kari 10 HP
Round 10: Geir 5 HP, Kari 0 HP
Winner: Geir after 10 rounds
```

In [25]:
```
#Warrior with a sword against Mage with a staff
sword = Weapon("Sword", Dice(12)) #deals 1-12 damage
staff = Weapon("Staff", Dice(6)) #deals 1-6 damage
battle = Battle(Warrior("Geir", weapon=sword), Mage("Kari", weapon=staff)) #both
winner = battle.simulate_battle()
for line in battle.log:
```

```
    print(line)
print(f"Winner: {winner.name} after {battle.round_number} rounds")
```

```
Round 1: Geir 85 HP, Kari 94 HP
Round 2: Geir 70 HP, Kari 87 HP
Round 3: Geir 55 HP, Kari 84 HP
Round 4: Geir 54 HP, Kari 81 HP
Round 5: Geir 53 HP, Kari 69 HP
Round 6: Geir 38 HP, Kari 64 HP
Round 7: Geir 38 HP, Kari 60 HP
Round 8: Geir 38 HP, Kari 48 HP
Round 9: Geir 23 HP, Kari 41 HP
Round 10: Geir 23 HP, Kari 31 HP
Round 11: Geir 23 HP, Kari 25 HP
Round 12: Geir 8 HP, Kari 20 HP
Round 13: Geir 8 HP, Kari 14 HP
Round 14: Geir 8 HP, Kari 3 HP
Round 15: Geir 8 HP, Kari 0 HP
Winner: Geir after 15 rounds
```

In [27]:
```
#Simulate 100 battles ro see who wins most often with weapons
wins = {"Geir": 0, "Kari": 0}
for battle_number in range(100):
    warrior = Warrior("Geir", weapon=Weapon("Sword", Dice(12))) # new warrior wi
    mage = Mage("Kari", weapon=Weapon("Staff", Dice(6))) #new mage with full hea
    winner = Battle(warrior, mage). simulate_battle()
    wins[winner.name] += 1
print(wins)
```

```
{'Geir': 37, 'Kari': 63}
```

In [28]:
```
# To warriors whose damage is lower than the armor can never hurt eachother
battle = Battle(Warrior("Weak A", damage=3), Warrior("Weak B", damage=3)) #3 - 5
winner = battle.simulate_battle() #stops after MAX_ROUNDS
print(f"Winner: {winner}, rounds played: {battle.round_number}")
```

```
Winner: None, rounds played: 1000
```

Results and discussion

Two normal characters lost 10 HP each round, and the hero won after 10 rounds with 10 HP left, because it attacks first. Warrior against Mage also lasted 10 rounds, and the warrior won with 5 HP left. With weapons the damage was random: in one battle the warrior won after 15 rounds, and in 100 battles the warrior won 37 times and the mage 63 times. MAX_ROUNDS stops battles that can never end, such as two warriors whose damage is lower than the armor.